

Exp 1 to 7:

Exp 1:

```
#include<stdio.h>
#include<string.h>
#define max 20
int main()
{
    int trans_table[4][2]={1,3},{1,2},{1,2},{3,3}};
    int final_state=2,i;
    int present_state=0;
    int next_state=0;
    int invalid=0;
    char input_string[max];

    printf("Enter a string:");
    scanf("%s",input_string);
    int l=strlen(input_string);
    for(i=0;i<l;i++)
    {
        if(input_string[i]=='a')
            next_state=trans_table[present_state][0];
        else if(input_string[i]=='b')
            next_state=trans_table[present_state][1];
        else
            invalid=1;
        present_state=next_state;
    }
    if(invalid==1)
```

```

{

printf("Invalid input");

}

else if(present_state==final_state)

printf("Accept\n");

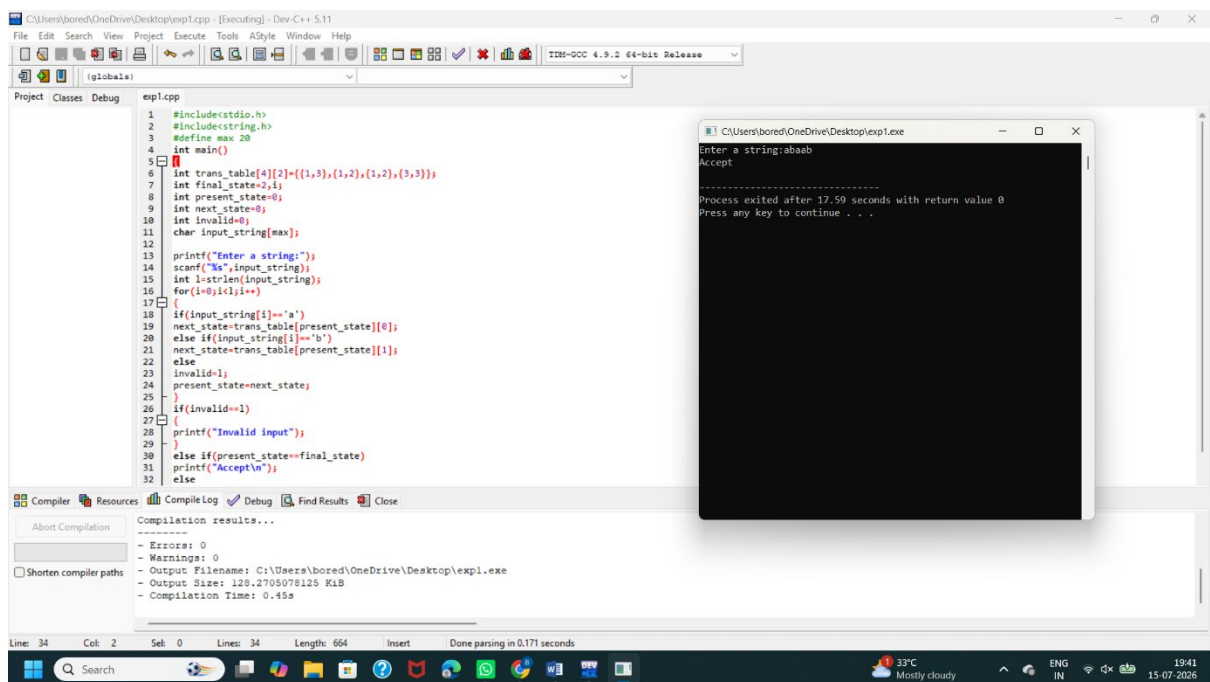
else

printf("Don't Accept\n");

}

```

Output:



Exp 2:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    int i, state = 0;
```

```
    printf("Enter the binary string: ");
```

```
scanf("%s", str);
```

```
for(i = 0; i < strlen(str); i++) {
```

```
    if(state == 0) {
```

```
        if(str[i] == '0')
```

```
            state = 0;
```

```
        else if(str[i] == '1')
```

```
            state = 1;
```

```
    }
```

```
    else if(state == 1) {
```

```
        if(str[i] == '0')
```

```
            state = 0;
```

```
        else if(str[i] == '1')
```

```
            state = 1;
```

```
    }
```

```
}
```

```
if(state == 1)
```

```
    printf("String Accepted.\n");
```

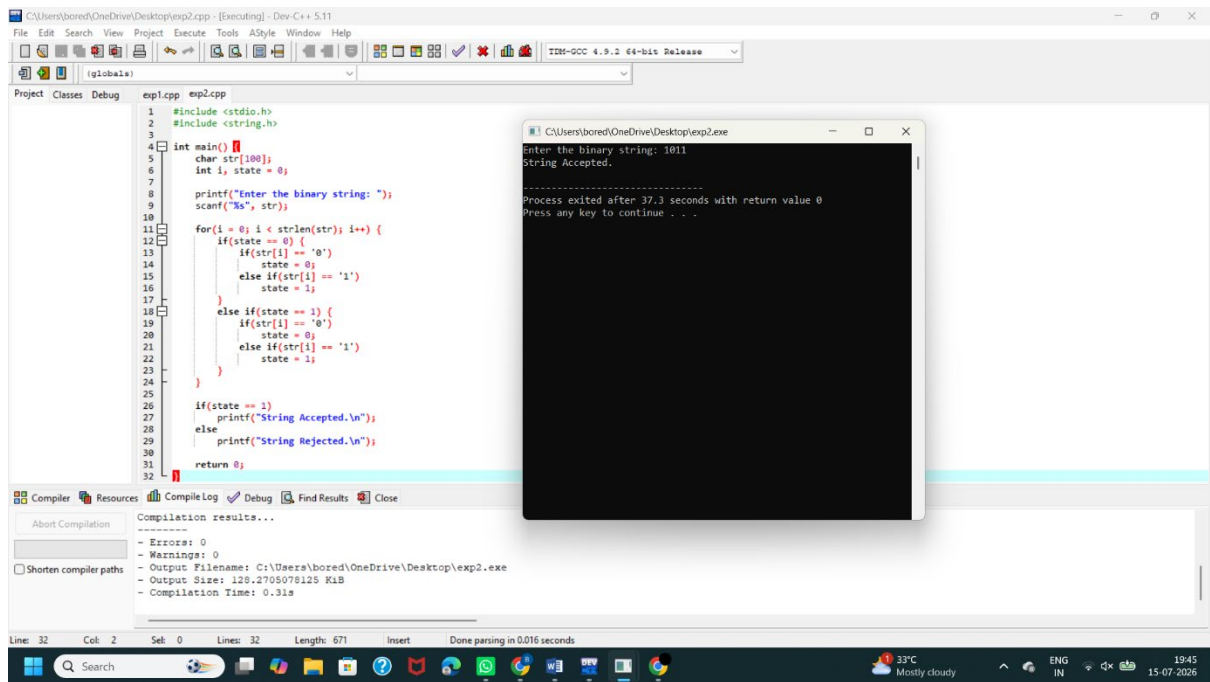
```
else
```

```
    printf("String Rejected.\n");
```

```
return 0;
```

```
}
```

Output:



Exp 3:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int trans_table[10][5][3];
```

```
char symbol[5],a;
```

```
int e_closure[10][10],ptr,state;
```

```
void find_e_closure(int x);
```

```
int main()
```

```
{
```

```
int i,j,k,n,num_states,num_symbols;
```

```
for(i=0;i<10;i++)
```

```
{
```

```
for(j=0;j<5;j++)
```

```
{
```

```
for(k=0;k<3;k++)
```

```
{
```

```
trans_table[i][j][k]=-1;
```

```
}
```

```

}
}

printf("How may states in the NFA with e-moves:");

scanf("%d",&num_states);

printf("How many symbols in the input alphabet including e :");

scanf("%d",&num_symbols);

printf("Enter the symbols without space. Give 'e' first:");

scanf("%s",symbol);

for(i=0;i<num_states;i++)
{
for(j=0;j<num_symbols;j++)

{

printf("How many transitions from state %d for the input %c:",i,symbol[j]);

scanf("%d",&n);

for(k=0;k<n;k++)

{

printf("Enter the transitions %d from state %d for the input %c :", k+1,i,symbol[j]);

scanf("%d",&trans_table[i][j][k]);

}

}

}

for(i=0;i<10;i++)

{

for(j=0;j<10;j++)

{

e_closure[i][j]=-1;

}

}

}

```

```

for(i=0;i<num_states;i++)
e_closure[i][0]=i;
for(i=0;i<num_states;i++)
{

if(trans_table[i][0][0]==-1)
continue;
else
{
state=i;
ptr=1;
find_e_closure(i);
}
}
for(i=0;i<num_states;i++)
{
printf("e-closure(%d)= {" ,i);
for(j=0;j<num_states;j++)
{
if(e_closure[i][j]!=-1)
{
printf("%d, ",e_closure[i][j]);
}
}
printf("}\n");
}
}

void find_e_closure(int x)
{

int i,j,y[10],num_trans;

```

```

i=0;

while(trans_table[x][0][i]!=-1)

{

y[i]=trans_table[x][0][i];

i=i+1;

}

num_trans=i;

for(j=0;j<num_trans;j++)

{

e_closure[state][ptr]=y[j];

ptr++;

find_e_closure(y[j]);

}

}

```

Output:

The screenshot shows a C++ IDE with the following components:

- Code Editor (exp1.cpp):** Contains the implementation of the e-closure algorithm. It defines a 3-state NFA and calculates the e-closure for each state based on the transitions.
- Output Window:** Displays the program's execution. It prompts the user for the number of states (3), input symbols (e), and transitions. It then shows the calculated e-closure sets:


```

e-closure(0)= { 0, 1, 2, }
e-closure(1)= { 1, 2, }
e-closure(2)= { 2, }

```
- Compiler Window:** Shows the compilation results, indicating 0 errors and 0 warnings. The output file is exp3.exe.

Exp 4:

```

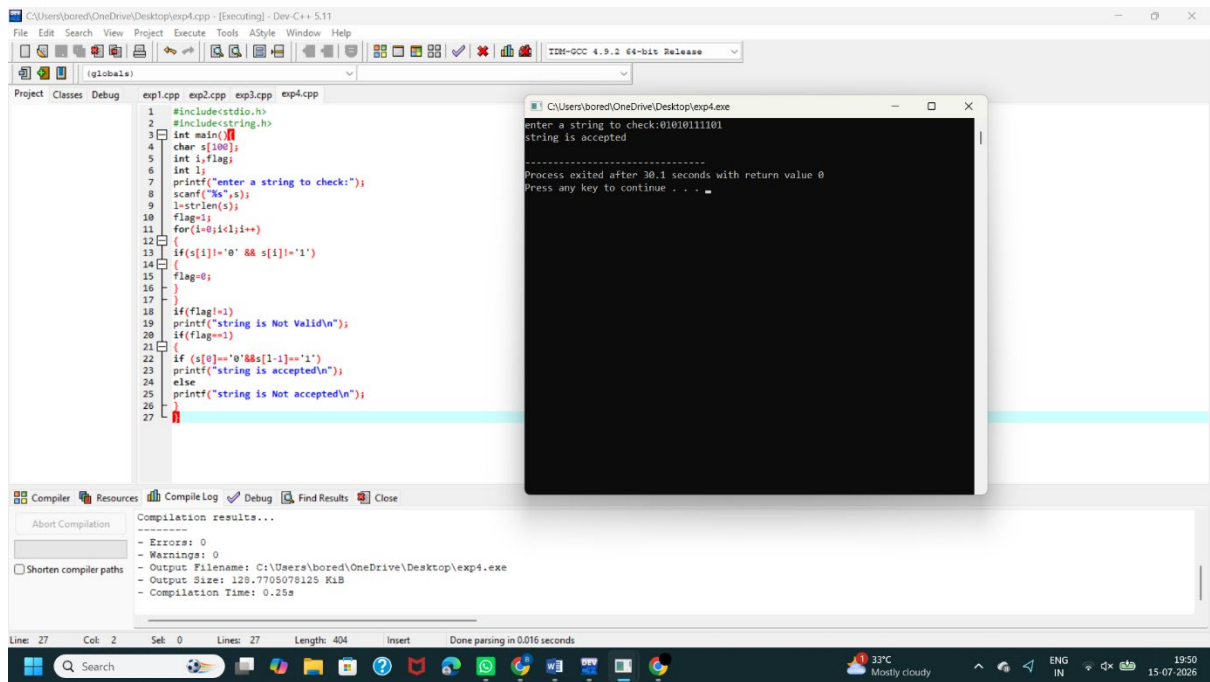
#include<stdio.h>

#include<string.h>

```

```
int main(){  
    char s[100];  
    int i,flag;  
    int l;  
    printf("enter a string to check:");  
    scanf("%s",s);  
    l=strlen(s);  
    flag=1;  
    for(i=0;i<l;i++)  
    {  
        if(s[i]!='0' && s[i]!='1')  
        {  
            flag=0;  
        }  
    }  
    if(flag!=1)  
        printf("string is Not Valid\n");  
    if(flag==1)  
    {  
        if (s[0]=='0'&&s[l-1]=='1')  
            printf("string is accepted\n");  
        else  
            printf("string is Not accepted\n");  
    }  
}
```

Output:



Exp 5:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100], stack[100];
```

```
    int top = -1, i, len;
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    len = strlen(str);
```

```
    i = 0;
```

```
    // Push all 0's onto the stack
```

```
    while (i < len && str[i] == '0') {
```

```
        stack[++top] = '0';
```

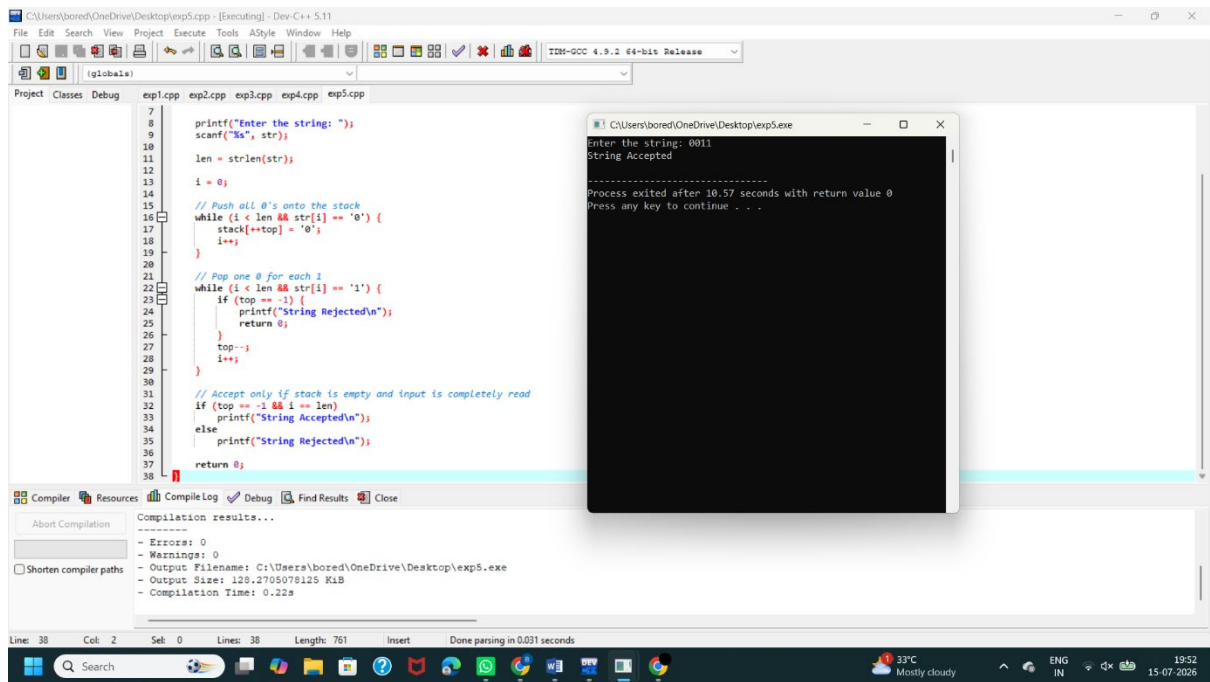
```
        i++;
    }

    // Pop one 0 for each 1
    while (i < len && str[i] == '1') {
        if (top == -1) {
            printf("String Rejected\n");
            return 0;
        }
        top--;
        i++;
    }

    // Accept only if stack is empty and input is completely read
    if (top == -1 && i == len)
        printf("String Accepted\n");
    else
        printf("String Rejected\n");

    return 0;
}
```

Output:



Exp 6:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100], stack[100];
```

```
    int top = -1, i = 0, len;
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", str);
```

```
    len = strlen(str);
```

```
    // Push two symbols for every 'a'
```

```
    while (i < len && str[i] == 'a') {
```

```
        stack[++top] = 'X';
```

```
        stack[++top] = 'X';
```

```
        i++;
```

```
}
```

```
// Pop one symbol for every 'b'
```

```
while (i < len && str[i] == 'b') {
```

```
    if (top == -1) {
```

```
        printf("String Rejected\n");
```

```
        return 0;
```

```
    }
```

```
    top--;
```

```
    i++;
```

```
}
```

```
if (top == -1 && i == len)
```

```
    printf("String Accepted\n");
```

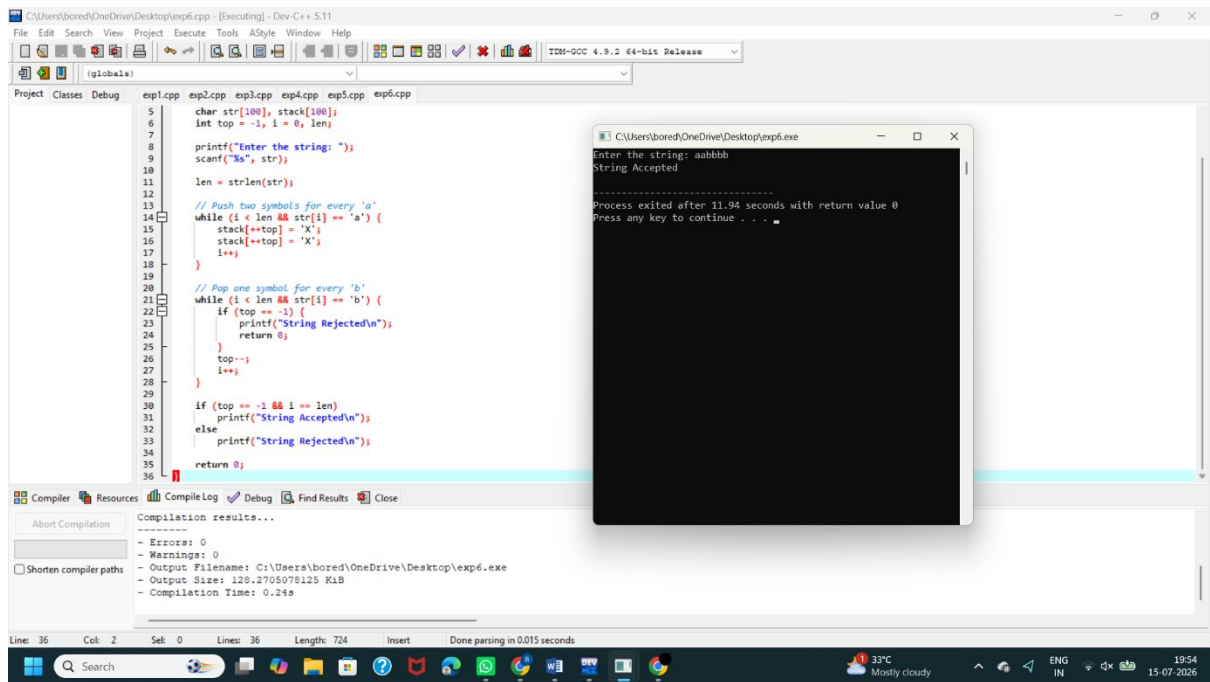
```
else
```

```
    printf("String Rejected\n");
```

```
return 0;
```

```
}
```

Output:



Exp 7:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char tape[100];
```

```
    int i, j, len;
```

```
    printf("Enter the string: ");
```

```
    scanf("%s", tape);
```

```
    len = strlen(tape);
```

```
    while (1) {
```

```
        i = 0;
```

```
        while (i < len && tape[i] != '0')
```

```
            i++;
```

```
    if (i == len)
        break;

    tape[i] = 'X';

    j = i + 1;
    while (j < len && tape[j] != '1')
        j++;

    if (j == len) {
        printf("String Rejected\n");
        return 0;
    }

    tape[j] = 'Y';
}

for (i = 0; i < len; i++) {
    if (tape[i] == '1') {
        printf("String Rejected\n");
        return 0;
    }
}

printf("String Accepted\n");
return 0;
}
```

Output:

